
DOCUMENTACIÓN DE OPERACIONES

Arquitectura de Software

SOLOServis

Plataforma de Comparación y Agregación de Productos y Servicios

Diseño de la arquitectura de microservicios: descomposición por dominio, justificación de las decisiones, alternativas descartadas y plan de implementación por etapas para el equipo de desarrollo.

Documento para el equipo Scrum

Agosto 2026

Contenido

1	Objetivo de este documento	2
2	Alcance	2
3	El modelo en una página	2
4	Diagrama de arquitectura	3
5	Servicios y responsabilidades	4
6	Las tres decisiones que definen el diseño	4
6.1	Productos y Servicios viven en un solo servicio	4
6.2	<i>Database per service</i>	4
6.3	Ingesta asíncrona vía <i>broker</i>	4
7	Trazabilidad con el Informe de Arquitectura de Software	6
7.1	Capas del informe y su realización	6
7.2	Requisitos y decisiones que los satisfacen	6
7.3	Brechas detectadas	8
8	Alternativas evaluadas y por qué se descartaron	8
9	Plan de implementación	10
9.1	Principio de arranque: Kubernetes no va primero	10
9.2	Sprint 0: los contratos antes que el código	10
9.3	Estructura del repositorio	10
9.4	Orden de construcción	10
9.5	Entorno local	10
9.6	Definición de terminado (DoD) por servicio	11
9.7	Controles de seguridad desde el inicio	11
9.8	Migración a Kubernetes	11
10	Riesgos a vigilar	12
11	Referencias	12

1. Objetivo de este documento

Este documento explica **cómo está diseñado SOLOServis por dentro y por qué se diseñó así**, para que todo el equipo comparta el mismo modelo mental antes de escribir código.

No es un documento de consulta teórica: la Sección 9 define el orden concreto de construcción, quién puede avanzar en paralelo y qué significa que un servicio esté terminado.

2. Alcance

Cubre la arquitectura de la **Fase 1** del proyecto: el flujo de búsqueda, comparación y redirección descrito en el *Modelo del Proceso de Negocio* y en el *Diagrama de Casos de Uso*, sobre el MER normalizado ya definido.

Quedan fuera: administración interna, moderación de reseñas y el detalle algorítmico del *scraping*, que se documentan por separado.

Las decisiones de diseño de este documento se derivan de los requisitos establecidos en el *Informe de Arquitectura de Software* [INF] y se validan contra el *Modelo del Proceso de Negocio* [MPN], el *Diagrama de Casos de Uso* [CU] y el *MER normalizado* [MER]. La correspondencia punto por punto está en la Sección 7; las referencias completas, al final del documento.

3. El modelo en una página

SOLOServis se divide en **cinco servicios que se despliegan de forma independiente**. Cada uno es dueño de un dominio de negocio completo: su propia lógica, su propia base de datos y su propia API.

La idea central que hay que retener es que el sistema tiene **dos flujos distintos** que nunca se mezclan:

Flujo síncrono (línea negra).

Lo que ocurre mientras el usuario navega. Entra por el *Ingress*, que enruta según la URL hacia el servicio correspondiente. El usuario espera la respuesta.

Flujo asíncrono (línea naranja).

La actualización del catálogo por *scraping*. Corre programado, por su cuenta, sin que ningún usuario esté esperando. El *scraper* no escribe en la base de datos: publica eventos.

Por qué importa: si el *scraping* se cae, se atrasa o falla, el usuario que está comparando productos no se entera. Y si el *scraper* llegara a ser comprometido, no tiene credenciales para escribir en ninguna base de datos.

4. Diagrama de arquitectura

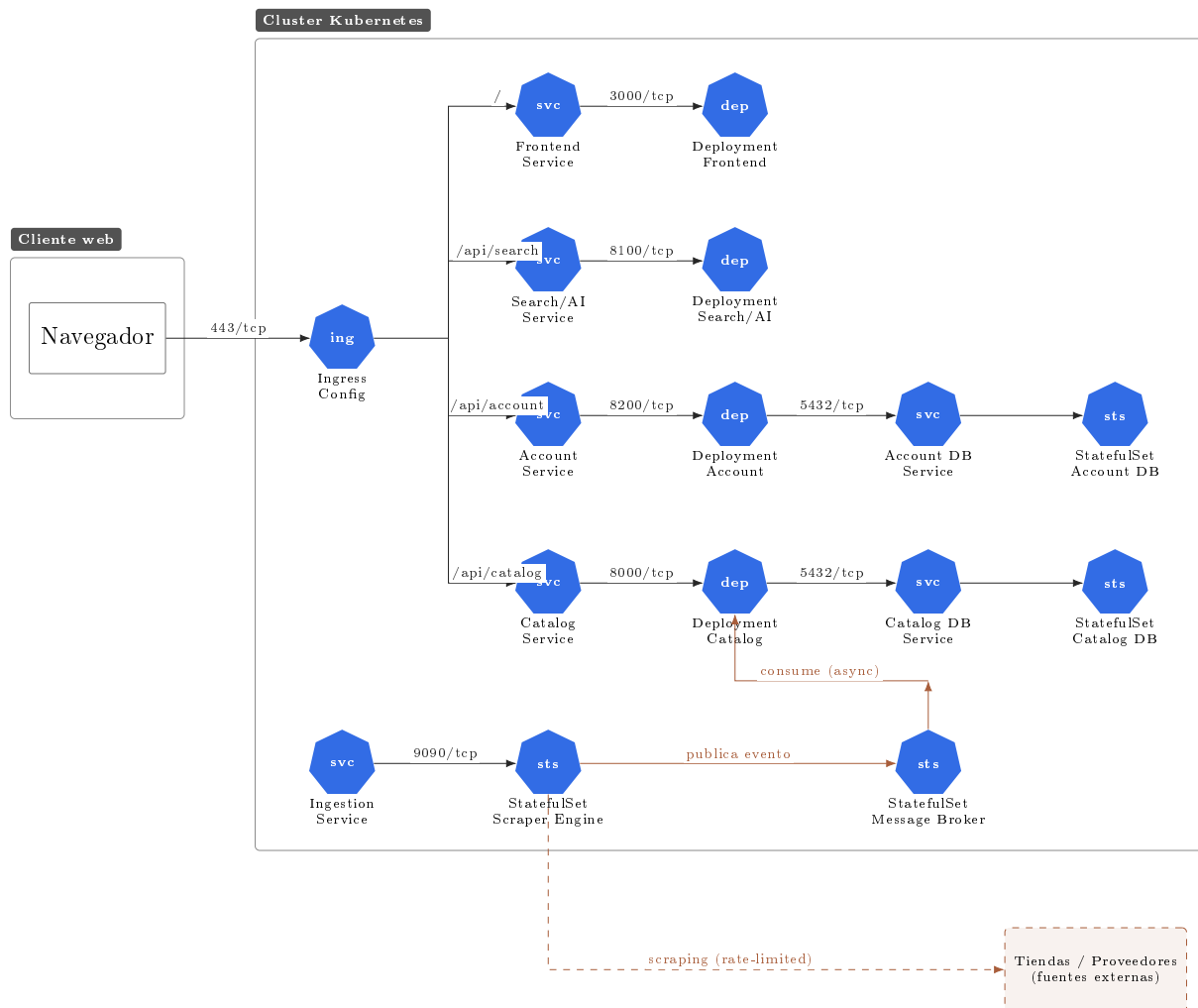


Figura 1: Arquitectura de despliegue de SOLOServis. En negro el tráfico síncrono del usuario; en naranja el flujo asíncrono de ingesta. Cada dominio con estado tiene su propia base de datos.

5. Servicios y responsabilidades

Servicio	Puerto	Responsabilidad	Entidades del MER
Frontend	3000	Sirve la SPA. Sin lógica de negocio.	—
Catalog	8000	Productos y servicios unificados: búsqueda, detalle, comparación, historial de precios, reseñas, tiendas.	PRODUCT, SERVICE, *_OFFER, *_COMPARISON, *_REVIEW, *_PRICE_HISTORY
Search/AI	8100	Interpreta la consulta: detecta intención, categoría, marca, rango de precio. Traduce lenguaje natural a filtros.	SEARCH_QUERY, SEARCH_QUERY_FILTER, AI_QUERY_PROCESS
Account	8200	Perfil del usuario y favoritos. Autocontenido.	USER_PROFILE, *_USER_INTEREST
Ingestion	9090	Orquesta el <i>scraping</i> programado y publica los datos recolectados como eventos.	DATA_SOURCE, SCRAPER_CONFIG, SCRAPE_RUN, SCRAPED_DATA

Tabla 1: Descomposición por dominio y correspondencia con el MER normalizado.

6. Las tres decisiones que definen el diseño

6.1 Productos y Servicios viven en un solo servicio

El diagrama de casos de uso [CU] concluye que ambos paquetes son **espejo funcional**: mismo patrón de flujo (buscar → seleccionar → comparar → ver detalle → ver precio/historial → ver tienda).

Separarlos en dos microservicios habría duplicado la misma lógica en dos bases de código, obligando a mantener en paralelo cada cambio. Un solo servicio parametrizado por tipo de ítem evita esa duplicación sin perder claridad de dominio.

6.2 Database per service

Cada servicio con estado tiene su propia base de datos y **ningún servicio accede a la base de otro**. Si Account necesita datos del catálogo, se los pide a Catalog por su API.

Esto materializa el requisito de **mantenibilidad y modularidad** [INF, RNF-1], que exige *decoupling* estricto entre el motor de *scraping*, el *pipeline* de IA y los *endpoints* de consulta.

Este es el punto que separa microservicios reales de un monolito distribuido. Una base compartida acopla los servicios por el esquema: cualquier cambio de tabla obliga a coordinar despliegues, y una consulta pesada de un servicio degrada a todos los demás. En seguridad, además, acota el radio de impacto: comprometer Catalog no entrega acceso a los datos de perfil de los usuarios.

Costo asumido: no hay *joins* entre dominios ni transacciones ACID que crucen servicios. Lo que antes era un *join* ahora es una llamada de red.

6.3 Ingesta asíncrona vía *broker*

El *scraper* no escribe en la base de datos del catálogo: publica un evento (`scrape.completed`) y Catalog lo consume. Esta decisión responde directamente al requisito no funcional de **CEA01**

[INF]: “*el proceso de scraping no debe bloquear la API de búsqueda*”.

El *scraper* es el único componente que ejecuta lógica sobre datos de fuentes externas no confiables; es el punto más expuesto del sistema. Si escribiera directo en la base de Catalog, tendría credenciales de escritura sobre todo el catálogo. Con el *broker* en medio, solo puede publicar mensajes en una cola: es Catalog –dueño de esos datos– quien decide qué valida y qué persiste.

Costo asumido: el sistema pasa a tener **consistencia eventual**. Un precio *scrapeado* no aparece al instante en el catálogo: hay un retraso de segundos o minutos. Para una plataforma de comparación es aceptable, pero la interfaz debe mostrar la marca de tiempo de la última actualización en vez de dar a entender que el precio es en tiempo real.

7. Trazabilidad con el Informe de Arquitectura de Software

Esta sección conecta cada elemento del diseño con el requisito que lo origina, para que el equipo pueda justificar cualquier componente del diagrama frente a la documentación del proyecto.

7.1 Capas del informe y su realización

El informe [INF] adopta una *Arquitectura Cliente-Servidor Desacoplada Basada en Servicios/Microservicios* de cuatro capas. La correspondencia con la Figura 1 es la siguiente:

Capa [INF]	Componente en el diseño	Observación
1. Presentación (Cliente)	Navegador → Frontend Service / Deployment	El informe menciona interfaz “web y/o móvil”; la Fase 1 cubre solo web.
2. API Gateway & Orquestación	Ingress Config (+ API Gateway)	El Ingress enruta, pero no orquesta. Ver brecha B1 .
3. Inteligencia Artificial	Search/AI Service (8100)	Coincidencia literal: el informe ya lo define como “microservicio especializado en NLP”.
4. Datos y Extracción	Catalog DB, Account DB, Ingestion Service, Scraper Engine, Message Broker	El informe pide motores “alimentados de forma asíncrona por la granja de <i>crawlers/scrapers</i> ”: eso es exactamente el <i>broker</i> .

Tabla 2: Capas del patrón arquitectónico [INF] y su realización en el despliegue.

7.2 Requisitos y decisiones que los satisfacen

Requisito [INF]	Dónde se resuelve
CEA01 — RNF de escalabilidad: el <i>scraping</i> no debe bloquear la API de búsqueda	Ingesta asíncrona vía <i>broker</i> (§6.3). El <i>scraper</i> publica eventos y no tiene credenciales de escritura sobre ninguna base de datos.
CEA01 — <i>rate limiting</i> y <i>anti-bot</i> de sitios origen	Arista <i>scraping</i> (<i>rate-limited</i>) de la Figura 1; control implementado en Ingestion Service.
CEA02 — la IA preprocesa la consulta y devuelve JSON estructurado en <800 ms	Search/AI Service (8100), con contrato POST /search/interpret (§8.2).
CEA02 — <i>fallback</i> a búsqueda por palabras clave si la IA no responde	Sin dueño asignado en el diseño actual. Ver brecha B1 .
RNF-1 — mantenibilidad: <i>decoupling</i> estricto entre <i>scraping</i> , IA y <i>endpoints</i> de consulta	Tres servicios independientes: Ingestion, Search/AI y Catalog, con bases de datos separadas (§6.2).
RNF-2 — tiempo de respuesta global <1,5 s	En riesgo con el enrutamiento actual. Ver brecha B1 .
RNF-4 — <i>rate limiting</i> por IP/usuario en el API Gateway	Punto único de autenticación y límite en el <i>gateway</i> (§8.7).
RF-4 — gestión de fuentes de <i>scraping</i> (panel interno)	Entidades DATA_SOURCE y SCRAPER_CONFIG en Ingestion Service; falta la interfaz. Ver brecha B4 .
Patrón Adapter — homogeneizar respuestas de distintos <i>scrapers</i>	Interno al Scraper Engine; no altera el despliegue.
Patrón Strategy — búsqueda vectorial, relacional o por palabras clave	Requiere un motor de búsqueda que hoy no está en el diagrama. Ver brecha B2 .

Tabla 3: Trazabilidad entre requisitos del informe y decisiones de arquitectura.

7.3 Brechas detectadas

El cruce anterior deja al descubierto cuatro puntos abiertos. No son errores del diagrama ni del informe por separado: son inconsistencias entre ambos que el equipo debe cerrar antes de codificar.

B1 — Un Ingress no es una capa de orquestación.

El informe define la Capa 2 como “API Gateway y Orquestación”, que “distribuye las solicitudes entre la IA y la base de datos”. Un **Ingress** de Kubernetes solo enruta por *path*. Con el enrutamiento actual, el navegador tendría que llamar primero a `/api/search` y después a `/api/catalog`, encadenándolo por su cuenta. Eso deja sin dueño el *fallback* de CEA02 y compromete el presupuesto de 1,5 s al duplicar los viajes de ida y vuelta. *Acción propuesta:* incorporar un **API Gateway / BFF** como servicio propio que reciba la búsqueda, invoque a Search/AI con *timeout*, aplique el *fallback* y consulte a Catalog, todo en una sola petición del cliente.

B2 — Falta el motor de búsqueda.

El informe especifica “Motor de Búsqueda / VectorDB: PostgreSQL / Elasticsearch” y el patrón Strategy contempla búsqueda vectorial. El diagrama solo tiene PostgreSQL. *Acción propuesta:* definir si la búsqueda será relacional (**pgvector** sobre la base existente) o requiere un **StatefulSet** de Elasticsearch, y en ese caso añadir cómo se actualiza el índice desde la ingesta.

B3 — El informe no cubre el dominio Cuenta.

El informe no contiene ningún requisito sobre perfil de usuario ni favoritos, pero el MER [MER] define `USER_PROFILE` y `*_USER_INTEREST`, y los casos de uso [CU] incluyen el paquete completo (CU-C1 a CU-C4). *Acción propuesta:* agregar un requisito CEA para el dominio Cuenta, o declarar explícitamente que queda fuera del alcance de la Fase 1.

B4 — El panel de gestión de fuentes no está representado.

El requisito funcional 4 pide un “panel interno para agregar, habilitar o pausar la extracción de determinados dominios web”. No aparece ni como *endpoint* en Ingestion ni como vista en el Frontend. *Acción propuesta:* definir los *endpoints* administrativos de Ingestion en el Sprint 0 y su control de acceso.

Inconsistencia de nomenclatura: el informe denomina al proyecto “SoloTodo para Productos y Servicios”, mientras que el resto de la documentación usa *SOLOServis*. Conviene unificar el nombre antes de la entrega.

8. Alternativas evaluadas y por qué se descartaron

Alternativa	Motivo del descarte
Monolito modular	Para el volumen real de la Fase 1 habría bastado y costaría menos operar; conviene decirlo con honestidad. Se descarta porque el proyecto exige aplicar microservicios y porque el <i>scraping</i> tiene un ciclo de vida genuinamente distinto (corre programado, falla aparte, consume recursos en ráfagas). De esta alternativa se rescata la disciplina de no fragmentar de más : cinco servicios, no doce.
Microservicios por capa técnica	Separar “frontend”, “backend” y “base de datos” como servicios distintos. Es el error más frecuente y el más fácil de detectar en una evaluación: un <i>Backend Service</i> único que contiene catálogo, cuentas y búsqueda es un monolito con otro nombre –un solo ciclo de despliegue, un solo punto de falla, una sola base compartida– y solo agrega latencia de red entre capas que antes se llamaban en memoria. El criterio correcto es separar por dominio , no por capa.
Microservicios finos (uno por entidad)	Un servicio por cada entidad del MER. Consultar el detalle de un producto –que según los casos de uso siempre incluye precio actual e historial («include»– requeriría tres o cuatro llamadas de red para lo que es una consulta con <i>joins</i> . Multiplica puntos de falla, latencia y superficie de ataque sin ganar nada: esas entidades cambian y se consultan juntas.
SOA con bus (ESB)	El bus se convierte en punto único de falla y en cuello de botella: toda la lógica de integración termina concentrada ahí. Para cinco servicios es una capa de indirección que no paga su costo.

Tabla 4: Opciones consideradas antes de fijar la arquitectura.

9. Plan de implementación

9.1 Principio de arranque: Kubernetes no va primero

La Figura 1 muestra el despliegue **objetivo**, no el punto de partida. Montar un clúster antes de tener los servicios funcionando agrega complejidad operativa (*manifests*, *Ingress*, *Secrets*, RBAC) en un momento donde el problema real es otro: definir bien los límites entre servicios.

El orden es: **primero los servicios corriendo en local con Docker Compose**, después la contenedorización estable, y solo al final los *manifests* de Kubernetes. La topología se conserva íntegra –mismos servicios, mismos puertos, mismas bases separadas–; lo que cambia es el orquestador.

9.2 Sprint 0: los contratos antes que el código

Antes de escribir lógica, definir la API de cada servicio (OpenAPI) y el esquema de los eventos. En microservicios **el contrato es la frontera**: si se define después del código, cada servicio termina exponiendo su modelo interno y el acoplamiento vuelve por la ventana.

- GET /catalog/items — búsqueda con filtros
- GET /catalog/items/{id} — detalle con precio actual e historial
- GET /catalog/stores/{id} — detalle de tienda y sus ítems
- POST /search/interpret — devuelve filtros estructurados
- GET /account/profile, GET|POST /account/favorites
- Evento `scrape.completed` — fuente, ítems, precios, *timestamp*

9.3 Estructura del repositorio

Se recomienda **monorepo**: una carpeta por servicio, cada una con su `Dockerfile` y sus dependencias.

```
soloservis/
  services/ {catalog, search-ai, account, ingestion, frontend}
  contracts/ # OpenAPI + esquemas de eventos
  docker-compose.yml
  k8s/      # manifests (etapa posterior)
```

Un repositorio por servicio es lo habitual en equipos grandes con despliegues independientes; para este equipo agrega fricción de coordinación sin beneficio.

9.4 Orden de construcción

El orden importa: cada servicio debe poder probarse **sin que los demás existan todavía**.

Las épicas 1 y 3 no tienen dependencias entre sí: dos personas o subequipos pueden avanzar en paralelo desde el primer sprint. El Frontend también, siempre que los contratos del Sprint 0 estén cerrados.

9.5 Entorno local

Un `docker-compose up` debe dejar corriendo los cinco servicios, dos PostgreSQL (`catalog_db` y `account_db`) y el *broker* –RabbitMQ es más simple de operar que Kafka para este caso.

Criterio de listo del entorno: una persona nueva clona el repositorio, ejecuta un comando y tiene el sistema funcionando sin configuración manual.

#	Épica	Por qué en esta posición	Depende de
1	Catalog Service	Es el núcleo: sin catálogo no hay nada que comparar. Partir con datos cargados a mano (<i>seeds</i>) para no depender del <i>scraper</i> .	—
2	Ingestion + <i>broker</i>	Reemplaza los <i>seeds</i> por datos reales. Aquí se valida el flujo asíncrono completo.	Épica 1
3	Account Service	Independiente de los anteriores: se puede desarrollar en paralelo desde el inicio.	—
4	Search/AI Service	Necesita un catálogo poblado para que la interpretación tenga contra qué probarse. Puede partir con reglas simples y evolucionar a NLP.	Épica 1
5	Frontend	En paralelo desde el inicio, contra las APIs del Sprint 0 (<i>mockeadas</i> si hace falta).	Contratos

Tabla 5: Orden de construcción y dependencias entre épicas.

9.6 Definición de terminado (DoD) por servicio

Un servicio se considera terminado cuando cumple **todos** estos puntos:

1. Su API responde según el contrato OpenAPI acordado en el Sprint 0.
2. Levanta con `docker-compose up` sin pasos manuales.
3. Tiene pruebas de sus casos de uso principales.
4. No accede a la base de datos de ningún otro servicio.
5. Sus credenciales vienen de variables de entorno, no del código.
6. Valida y sanitiza toda entrada externa antes de persistirla.

9.7 Controles de seguridad desde el inicio

No son un agregado posterior; varios son mucho más caros de incorporar después:

- **Credenciales fuera del código.** Variables de entorno en desarrollo; *Secrets* de Kubernetes o Vault en despliegue. Nunca en el repositorio ni en la imagen.
- **Validación estricta en el consumidor de eventos.** Catalog debe tratar todo lo que viene del *scraper* como entrada no confiable: tipos, rangos de precio, longitudes, sanitización.
- **Rate limiting en el *scraper*,** por respeto a las fuentes externas y para no exponerse a bloqueos.
- **Autenticación en el *API Gateway*,** no en cada servicio: un solo punto donde se valida el *token*.
- **Escaneo de imágenes** (Trivy o Gype) antes de desplegar, especialmente la del *scraper*.

9.8 Migración a Kubernetes

Recién cuando el sistema funcione en Compose. La traducción es directa: cada servicio pasa a `Deployment + Service`, las bases a `StatefulSet` con `PersistentVolumeClaim`, el *scraper* a `CronJob`, y el enrutamiento por *path* a un `Ingress`.

El punto más valioso de esta etapa son las **NetworkPolicies**: denegar por defecto todo el tráfico entre *pods* y habilitar explícitamente solo los caminos de la Figura 1; en particular, que la ingesta sea lo único con salida a internet y que los servicios de negocio no tengan *egress* externo.

Advertencia: sin esas reglas, el aislamiento que se ve en el diagrama **no existe** en el clúster real. Es solo un dibujo.

10. Riesgos a vigilar

Search/AI puede quedar demasiado delgado.

Si termina siendo solo un *parser* de palabras clave que delega todo a Catalog, deja de justificar su existencia como servicio separado y solo agrega un salto de red. Conviene revisarlo a mitad del desarrollo: si no tiene lógica propia sustantiva, fusionarlo con Catalog y documentar la decisión.

Consistencia eventual mal comunicada.

Si la interfaz muestra precios sin indicar cuándo se actualizaron, el usuario asumirá tiempo real y la plataforma perderá credibilidad cuando el precio no coincida con la tienda.

El *broker* es un punto único de falla.

Si se cae, la ingesta se detiene por completo. Para la Fase 1 es un riesgo aceptable –el catálogo sigue sirviendo datos anteriores– pero debe estar documentado y monitoreado.

11. Referencias

Documentos fuente de este diseño. Los códigos entre corchetes son los usados como citas a lo largo del texto.

[INF] *Informe de Arquitectura de Software: Plataforma “SoloTodo para Productos y Servicios”*. Universidad Católica de Temuco, Ingeniería Civil en Informática. Contiene requisitos CEA01–CEA02, requisitos funcionales y no funcionales generales, patrones de diseño aplicados y el patrón arquitectónico de cuatro capas.

[CU] *Diagrama de Casos de Uso — SOLOServis*. Agosto 2026. Paquetes Productos, Servicios y Cuenta; relaciones «include» y «extend».

[MPN] *Modelo del Proceso de Negocio — SOLOServis*. Agosto 2026. Flujo por carriles (Usuario, Plataforma, Tienda/Proveedor), etapas 1–6 y sus entidades asociadas.

[MER] *MER normalizado — Informe v1.1*. Modelo entidad-relación sobre el que se define la propiedad de datos de cada servicio.

Documento vivo: actualizar ante cualquier cambio de límites entre servicios o de contratos de API.